

C.A.S.C.O.: A Smart Helmet Prototype for Fall Detection, Event Capture, and Real-Time Alerts

Josué López, Student Member, IEEE
Jorge Larios, Student Member, IEEE
Francisco Reyes, Student Member, IEEE
Job N. Acosta, Member, IEEE

*Faculty of Engineering
Universidad Evangélica de El Salvador
San Salvador, El Salvador*

josuerlb1110@gmail.com, romerojorge0504@gmail.com,
Reeyes888@gmail.com, jobacosta@ieee.org

Abstract—This paper presents C.A.S.C.O. (Casco Autónomo con Sensores de Control y Observación), an autonomous IoT safety-helmet prototype designed to improve post-fall awareness in industrial and construction environments. The system combines an ESP32-WROOM-32, an MPU6050 inertial measurement unit, an ESP32-CAM module, and a cloud-connected software stack to detect fall events, preserve short video evidence, and notify supervisors in real time. A three-stage finite-state machine evaluates freefall, impact, and post-impact stillness using empirically selected accelerometer and gyroscope thresholds. When a fall is confirmed, the sensor node sends an ESP-NOW trigger to the camera node, which locks a seven-second pre-event JPEG buffer in PSRAM, records seven seconds after the event, uploads the resulting MJPEG clip to backend storage, and emits alerts through Socket.IO. Preliminary testing over 50 fall-like trials and 80 non-fall trials produced 92.0% sensitivity and 92.5% specificity. Communication tests showed 50/50 successful ESP-NOW pairing attempts with an average 5 s pairing completion time, while the video pipeline averaged two lost frames per recorded event, 15 s upload time, and 5 h continuous runtime from a 5 V, 5000 mAh battery under repeated accident-video recording. The results support the feasibility of the low-cost architecture while showing that further classifier tuning is required before safety-critical deployment.

Index Terms—ESP-NOW, ESP32-CAM, fall detection, industrial safety, Internet of Things, MPU6050, real-time alerts, smart helmet.

I. INTRODUCTION

Falls remain a major safety risk in construction work. The U.S. Bureau of Labor Statistics reported 1,034 workplace deaths in the private-sector construction industry in 2024, including 389 deaths from falls, slips, and trips (37.6% of construction fatalities); 373 of those 389 (95.9%) were falls to a lower level [1]. OSHA's National Safety Stand-Down continues to treat fall prevention as a campaign priority [2]. NIOSH similarly identifies construction as a sector with a disproportionate share of fatal slips, trips, and falls [3]. These data motivate systems that can detect incidents quickly and help supervisors reconstruct what happened after an emergency.

C.A.S.C.O. addresses this need through a smart safety helmet that detects a probable fall, captures context video

from before and after the event, and reports the event to web and mobile clients. The design emphasizes low-cost hardware, free-tier cloud deployment, and a simple first-use workflow: the worker powers the helmet, the camera module joins Wi-Fi once through a captive portal, and the system thereafter operates autonomously. This user-experience goal came from the project plan requirement that workers should not have to enter technical IP addresses or configure the sensor node manually.

The main contribution of this work is an end-to-end smart-helmet prototype that combines local threshold-based fall detection, ESP-NOW-based inter-device triggering, PSRAM-based pre-event video buffering, cloud event storage, and real-time web/mobile supervisor alerts. Unlike alert-only fall-detection systems, the proposed prototype preserves visual context before and after the incident while avoiding direct inbound network access to the helmet. This paper reports the implementation and integration status honestly as a prototype, not as certified safety equipment.

II. STATE OF THE ART

Wearable fall-detection research commonly relies on inertial sensors such as accelerometers and gyroscopes. Ramachandran and Karuppiah survey wearable fall-detection systems and note the role of both threshold-based and machine-learning approaches in distinguishing falls from normal activity [4]. Wu *et al.* report a waist-worn quaternion detector at 97.1% sensitivity and 98.3% specificity against a 91.6%/88.7% acceleration-threshold baseline [5]. Table I places those figures beside CareFall's SMV-threshold versus random-forest results on the Erciyes wrist dataset [6] and Qian *et al.*'s 20-volunteer IoT wearable [7]. The studies are not interchangeable with C.A.S.C.O.; on-helmet learning is not claimed here.

Smart-helmet research for industrial environments often focuses on environmental risk, worker monitoring, and hazard communication. Campero-Jurado *et al.* present a smart helmet prototype for Industrial Internet of Things risk evaluation,

TABLE I
PUBLISHED FALL-DETECTION RESULTS VERSUS THIS PROTOTYPE (NOT A HEAD-TO-HEAD TEST)

System	Method / Placement	Sens.	Spec.	Acc.	<i>n</i> / Set
Wu <i>et al.</i> [5]	Quaternion / waist	97.1	98.3	–	dataset in [5]
Wu <i>et al.</i> [5]	Accel. threshold / waist	91.6	88.7	–	dataset in [5]
CareFall [6]	SMV threshold / Erciyes wrist	100	68.4	77.3	Erciyes
CareFall [6]	Random forest / Erciyes wrist	98.9	96.7	98.4	Erciyes
Qian <i>et al.</i> [7]	Multilevel threshold / wearable	95.25	94.5	94.88	20 volunteers
C.A.S.C.O. (this work)	Threshold FSM / helmet	92.0	92.5	92.3	3 students; 50+80

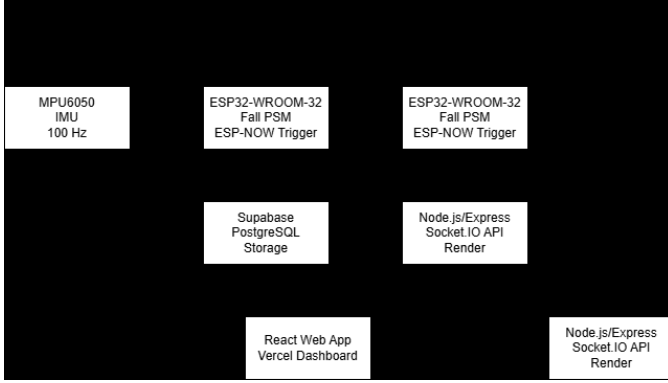


Fig. 1. C.A.S.C.O. system architecture.

demonstrating that helmets can serve as platforms for sensing and near real-time safety assessment [8]. C.A.S.C.O. differs from many alert-only designs by combining helmet-mounted inertial detection with pre/post-event video evidence and cloud-based supervisor notification. Its current novelty is therefore primarily architectural and integrative; formal novelty claims about detection reliability require controlled experiments.

III. SYSTEM ARCHITECTURE

The C.A.S.C.O. prototype is divided into four main layers: an inertial sensing node, an edge camera node, a backend and storage layer, and web/mobile user interfaces. Fig. 1 shows the end-to-end architecture. The sensor node is based on an ESP32-WROOM-32 connected to an MPU6050 over I2C. It samples acceleration and angular velocity at approximately 100 Hz and runs the fall-detection finite-state machine. When a fall is confirmed, it sends a compact trigger message to the ESP32-CAM using ESP-NOW.

The ESP32-CAM is responsible for image capture, event recording, upload, and streaming. During normal operation it maintains a circular buffer of JPEG frames in PSRAM. At the time of a fall trigger, it preserves the last seven seconds of pre-event imagery and captures seven seconds of post-event frames. It then exports the event as an MJPEG clip, uploads it through HTTPS, and sends a backend event notification. The backend runs on Node.js/Express with Socket.IO for real-time communication, PostgreSQL for event/device data, and Supabase Storage for clips and snapshots.

IV. EMBEDDED HARDWARE AND COMMUNICATION DESIGN

A. Sensor Node

The sensor node uses an ESP32-WROOM-32 and an MPU6050 inertial measurement unit. The MPU6050 integrates a 3-axis accelerometer and a 3-axis gyroscope in a single 6-axis motion device [9]. In the current prototype, the accelerometer is configured for $\pm 2g$, the gyroscope for ± 250 deg/s, and the digital low-pass filter for 20 Hz. The sampling loop uses an approximate 10 ms delay, producing an effective rate near 100 Hz.

The fall detector computes acceleration magnitude and gyroscope magnitude from the raw sensor values. It does not depend on Wi-Fi credentials or a router IP address. The only nonvolatile peer information stored by the sensor node is the ESP32-CAM MAC address obtained during ESP-NOW pairing. This keeps the WROOM firmware focused on deterministic local sensing and avoids user-facing configuration complexity.

B. Camera Node

The camera node uses an AI-Thinker ESP32-CAM with an OV2640 camera, 4 MB PSRAM, and 4 MB flash. The camera is configured for QVGA resolution at 320×240 pixels, JPEG quality 12, and two frame buffers. PSRAM is used to hold the circular pre-event buffer and the combined pre/post-event frame set during event capture.

The ESP32-CAM also manages Wi-Fi setup. On first use it launches a captive portal named CASCO-Setup using WiFi-Manager and displays the device identifier. After joining the user network, it generates a CASCO-XXXXXX identifier from the MAC suffix, auto-registers with the backend, and stores the returned device API token in nonvolatile storage. This design centralizes Wi-Fi and Internet communication in the camera node while leaving the WROOM node independent from network setup.

C. ESP-NOW Pairing and Triggering

ESP-NOW was selected for communication between the sensor and camera nodes. Espressif defines ESP-NOW as a connectionless Wi-Fi communication protocol in which application data is transmitted directly between devices without a conventional Wi-Fi connection [10]. Espressif also notes that ESP-NOW can operate directly device-to-device without requiring a router to forward packets [11]. These properties



Fig. 2. Three-stage fall-detection finite-state machine.

are suitable for a helmet architecture in which the sensor node should not depend on router association or an IP address.

The current pairing flow uses a broadcast byte and an acknowledgment byte. The camera node broadcasts 0xAA on the configured channel until the WROOM node receives it, stores the sender MAC address, registers it as a peer, and replies with 0xBB. On later restarts, the WROOM loads the stored camera MAC address and sends 0xBB again to restore the paired state. A confirmed fall is transmitted as a compact FallMessage containing a boolean fall flag and a floating-point severity value.

A key engineering correction was moving expensive operations out of the ESP-NOW receive callback. The callback runs in a Wi-Fi task context; performing HTTP requests or heavy PSRAM work inside the callback caused instability. The corrected design only updates volatile flags inside the callback, while the main loop performs event capture and backend communication.

V. FALL DETECTION ALGORITHM

The fall-detection algorithm is a three-stage finite-state machine, shown in Fig. 2. The machine transitions from IDLE to FREEFALL_DETECTED when acceleration magnitude drops below 0.5 g. If an impact greater than 2.5 g occurs within the configured time window and the gyroscope magnitude changes sharply, the state advances to IMPACT_DETECTED. The algorithm then requires approximately 1.5 s of post-impact stillness before confirming the fall and sending the alert.

The firmware uses the current acceleration magnitude, $|a|$, and the current and previous gyroscope magnitudes to estimate whether the motion sequence is consistent with a fall. The severity value transmitted to the camera node is defined as the measured impact acceleration divided by the 2.5 g impact threshold. This simple ratio allows the backend and clients to treat events with stronger impacts differently in later versions without increasing the ESP-NOW message size.

This threshold-based method was chosen for transparency and ease of calibration. The thresholds were selected empirically during prototype development by observing the intended freefall-impact-stillness sequence and reducing immediate false triggers from ordinary handling. The method remains a prototype baseline rather than certified safety equipment; the validation results in Section VIII show that additional

threshold tuning or learning-based classification is needed to reduce missed fall events.

VI. VIDEO BUFFERING AND CLOUD ALERT PIPELINE

The ESP32-CAM maintains a 70-frame circular JPEG buffer at 10 FPS, representing seven seconds of pre-event video. On a fall trigger, the firmware locks and copies this pre-buffer, then records an additional 70 post-event frames. The total event clip therefore contains 140 frames, approximately 14 seconds of incident context. The prototype estimates about 1.4 MB of PSRAM use per full event frame set.

The prototype stores the event as an MJPEG sequence rather than H.264. This is a practical embedded design choice: the ESP32-CAM can capture JPEG frames, but it does not provide hardware H.264 encoding. MJPEG keeps the implementation simple and compatible with the frame data already available from the camera driver, at the cost of larger storage size compared with modern inter-frame video codecs.

In addition to event clips, the camera node supports two forms of live viewing. A local MJPEG stream is available on the same network through the /stream route, while remote viewing is implemented as a pseudo-stream: the camera posts raw JPEG frames to the backend at 5 FPS, and the backend relays them to joined clients via Socket.IO. This avoids direct inbound connections to the helmet and works behind common network address translation.

The backend uses Node.js and Express routes for authentication, device management, event creation, file uploads, snapshots, and frame streaming. User clients authenticate with JWTs, while devices authenticate with API tokens. PostgreSQL tables store users, devices, and events. Socket.IO rooms are used to isolate real-time traffic by device identifier; the Socket.IO server API supports emitting events only to clients that have joined a specified room [12]. Supabase Storage is used for event clips and snapshots [13], and Supabase documentation distinguishes public and private storage models, including signed URLs for limited-time access [14].

Two client interfaces were implemented. The React web dashboard provides login, registration, device linking, device deletion, live frames, and an event list. The Flutter Android app mirrors the same core workflow, stores the JWT locally, connects to Socket.IO, and displays frame updates using Image.memory with gapless playback to reduce flicker.

VII. EXPERIMENTAL METHODOLOGY

A controlled prototype validation was conducted to address detection reliability, false alerts, communication behavior, video stability, upload delay, and runtime. The evaluation separated fall-like events from non-fall activities so that true positives, false negatives, true negatives, and false positives could be reported separately. Tests were performed with the helmet worn on a padded floor, indoors and outdoors. Those sessions were not labeled as a factor, so metrics cannot be split by environment. Three undergraduate student volunteers of similar height performed the trials; none was a construction worker, and no authentic job-site fall from height was

TABLE II
PROTOTYPE CONFIGURATION PARAMETERS

Parameter	Value
IMU sampling rate	approx. 100 Hz
Fall sequence	freefall, impact, stillness
Pre-event buffer	7 s, 70 JPEG frames
Post-event recording	7 s, 70 JPEG frames
Total clip length	approx. 14 s, 140 frames
Remote stream rate	5 FPS
Camera resolution	QVGA, 320 × 240
ESP-NOW channel	11, configurable
Backend deployment	Render free-tier service
Storage/database	Supabase Storage and PostgreSQL

recorded. Ground truth was assigned manually for each trial before the firmware output was compared with the expected fall or non-fall label.

The fall-like set contained 50 trials: 20 forward-fall trials, 10 lateral-fall trials, 15 backward-fall trials, and 5 helmet drop/impact simulations. The non-fall set contained 80 trials representing ordinary helmet and worker motions; the observed false alerts were associated mainly with sudden strong movements and rough helmet placing or dropping. Communication performance was evaluated with 50 ESP-NOW pairing attempts, and the video and power tests measured frame loss, stream interruption during recording, upload time, and continuous runtime. The tests should be interpreted as validation of the implemented prototype thresholds, not as final field validation or certification evidence.

The primary classifier metrics were sensitivity $TP/(TP + FN)$, specificity $TN/(TN + FP)$, false-alert rate $FP/(FP + TN)$, accuracy $(TP + TN)/N$, precision $TP/(TP + FP)$, and F1-score. ESP-NOW timing was reported as pairing completion time; fall-trigger delivery latency after pairing was not isolated as a separate measurement. Because the tests were conducted on a prototype rather than a certified safety device, the results are interpreted as preliminary performance indicators rather than final compliance evidence.

VIII. RESULTS AND DISCUSSION

The implemented prototype currently supports fall detection, automatic ESP-NOW pairing, automatic device registration, Wi-Fi configuration through a captive portal, PSRAM-based pre/post-event capture, clip upload, real-time Socket.IO alerts, remote frame streaming at 5 FPS, a deployed web dashboard, and a Flutter Android application. Table II summarizes the configuration parameters, Table III reports the fall-scenario and classifier results, and Table IV summarizes communication, video, upload, and power measurements.

The fall-like validation set produced 46 true positives and 4 false negatives in 50 trials, corresponding to a sensitivity of 92.0% and an average detection latency of 2 s. Each of the four fall scenarios contributed one missed detection. Forward-fall trials achieved the highest scenario sensitivity at 95.0%, followed by backward falls at 93.3%, lateral falls at 90.0%, and helmet drop/impact simulations at 80.0%. The non-fall set produced 74 true negatives and 6 false positives in 80 trials,



Fig. 3. Physical C.A.S.C.O. smart-helmet prototype used during controlled fall-like and non-fall validation trials.

corresponding to a specificity of 92.5% and a false-alert rate of 7.5%. Across both sets, the aggregate accuracy was 92.3%, with precision of 88.5% and an F1-score of 90.2%.

The main observed cause of false negatives was that the post-impact stillness condition was not always satisfied after a fall-like motion, causing the finite-state machine to reject some events even when freefall and impact behavior had occurred. The false positives were mainly associated with sudden strong non-fall movements and rough helmet placing or dropping, which can partially resemble the freefall-impact pattern. These observations suggest that future tuning should analyze each state-machine transition separately and may benefit from adaptive thresholds or TinyML-based classification.

Communication and video tests showed that the ESP-NOW path completed 50 of 50 pairing attempts, with an average pairing completion time of 5 s. This value refers to pairing/connection setup rather than measured fall-trigger delivery after pairing. Recorded event clips lost an average of two frames, remote stream viewing lost an average of two seconds while event recording was active, and 14-second MJPEG clips uploaded in an average of 15 s. The fall alert is generated before clip upload completes, so upload time describes clip availability rather than the first supervisor notification; end-to-end client alert latency was not isolated as a separate measurement. The power stress test used a 5 V, 5000 mAh battery, corresponding to a nominal energy capacity of approximately 25 Wh. Based on a heavy-load current draw of roughly 0.8–1.0 A during recording, upload, and streaming activity, the theoretical operating range is approximately 5–6.25 h. The

TABLE III
CONTROLLED VALIDATION AND FALL-SCENARIO RESULTS

Scenario	Trials	Result Counts	Primary Metric
Forward fall	20	TP=19, FN=1	Sensitivity 95.0%
Side/lateral fall	10	TP=9, FN=1	Sensitivity 90.0%
Backward fall	15	TP=14, FN=1	Sensitivity 93.3%
Helmet drop/impact	5	TP=4, FN=1	Sensitivity 80.0%
Fall-like total	50	TP=46, FN=4	Sensitivity 92.0%; avg latency 2 s
Non-fall activities	80	TN=74, FP=6	Specificity 92.5%; false-alert rate 7.5%
Combined classifier	130	Correct=120, errors=10	Accuracy 92.3%; precision 88.5%; F1-score 90.2%

TABLE IV
SYSTEM PERFORMANCE MEASUREMENTS

Subsystem	Test Scope	Measured Result
ESP-NOW	50 pairing attempts	50/50 successful; avg pairing completion time 5 s
Recorded video	Event recording under accident-video load	Avg 2 frames lost per recorded clip
Remote stream	Live pseudo-stream while recording	Avg 2 s video loss during event recording
Upload pipeline	14 s MJPEG event clip	Avg upload time 15 s
Power	5 V, 5000 mAh battery under repeated accident-video recording	5 h measured runtime; theoretical heavy-load estimate 5–6.25 h

measured stress-test result of 5 h under repeated accident-video recording therefore aligns with the conservative end of the calculated runtime range.

Indoor and outdoor sessions occurred, but the event log was not stratified. No per-environment confusion matrix exists, and none is invented here. Future trials must treat environment as an explicit factor.

IX. PRIVACY, SECURITY, SCALABILITY, AND DEPLOYMENT CONSIDERATIONS

Because C.A.S.C.O. captures workplace video, privacy and access control are central deployment concerns. In the current prototype, video recording is event-triggered rather than continuous, and public clip URLs are used only to simplify development testing. A production deployment should use private storage buckets, signed URLs, role-based access control, retention limits, and explicit worker consent policies. These controls are necessary to ensure that incident reconstruction does not become continuous worker surveillance.

The backend already separates user authentication from device authentication: users authenticate with JWTs, while devices authenticate with API tokens. Future revisions should define token expiration, token rotation, administrator roles, audit logs, and device unlinking behavior. Clips should be encrypted in transit through HTTPS and protected at rest through private storage policies. Supervisors should only access devices assigned to their account or organization, and incident clips should expire or be deleted according to a documented retention policy.

The intended deployment is one WROOM plus ESP32-CAM pair per worker. ESP-NOW provides the local trigger; the camera Wi-Fi uplink is the WAN bottleneck. The prototype was not load-tested beyond a single paired unit, so cluster capacity and roaming are unmeasured. ESP-NOW frames are not end-to-end encrypted, and there is no hardware root of trust. Encrypted radios, signed firmware, least-privilege clip

retention, and a data-protection impact assessment remain requirements, not implemented features.

X. ENGINEERING CHALLENGES AND DESIGN DECISIONS

Several implementation issues shaped the final architecture. First, ESP-NOW communication required channel alignment between the WROOM node and the camera node. Second, HTTPS requests from the camera required WiFiClientSecure handling. Third, the /stream/frame endpoint had to be registered before JSON body parsing in Express so that raw JPEG request bodies were not consumed by middleware. Fourth, the device identifier had to be generated after Wi-Fi initialization to avoid a zeroed MAC-derived identifier.

The project also made explicit design tradeoffs. ESP-NOW replaced local HTTP between microcontrollers because it avoids IP discovery and reduces user configuration. PSRAM buffering replaced SD-card dependence because it reduces mechanical and storage-management complexity. WebSocket relaying replaced direct camera streaming for remote viewing because it can operate behind NAT. Finally, the prototype uses free-tier infrastructure to meet the project constraint that the system remain free to deploy during development.

XI. LIMITATIONS AND FUTURE WORK

The current prototype has important limitations. Although initial validation provided measurable results, the fall detector still missed 4 of 50 fall-like events, producing an 8.0% miss rate, and generated 6 false alerts in 80 non-fall trials, producing a 7.5% false-alert rate. The results establish a quantitative prototype baseline but indicate that the detector is not yet reliable enough for safety-critical deployment. The false-negative observations suggest that the stillness stage should be retuned or complemented by additional features, while the false positives indicate that abrupt handling and rough helmet placement need stronger rejection logic. The event clip format is MJPEG rather than a compressed inter-frame video codec,

which increases storage and bandwidth requirements. Remote streaming still loses about two seconds during event recording, and public clip URLs simplify the prototype but are not ideal for sensitive industrial video data.

Future development should prioritize reducing false negatives through improved threshold tuning, larger field datasets, and TinyML-based fall detection with TensorFlow Lite Micro—including a same-trace learning baseline that this paper does not claim. It should also add charging electronics, push notifications through Firebase Cloud Messaging, binary streaming, private storage with signed URLs, iOS support, OTA firmware updates, and longer-term testing under realistic construction-site motions such as crouching, climbing, jumping, helmet handling, and repeated impacts. A future protocol should recruit at least 15 construction workers spanning height, sex, and trade; label environment as a factor; and report per-stratum metrics. Until that study exists, C.A.S.C.O. should be read as a feasibility prototype.

XII. CONCLUSION

C.A.S.C.O. demonstrates a practical smart-helmet architecture for detecting probable worker falls, preserving video context, and notifying supervisors through web and mobile interfaces. The prototype combines low-cost ESP32 hardware, inertial sensing, ESP-NOW communication, PSRAM video buffering, cloud storage, and Socket.IO real-time delivery. In the preliminary validation set, the prototype achieved 92.0% sensitivity, 92.5% specificity, 92.3% aggregate accuracy, 88.5% precision, and a 90.2% F1-score, together with 50/50 ESP-NOW pairing success with 5 s average pairing completion time, 15 s average video upload time, and 5 h measured runtime from a 5 V, 5000 mAh battery under repeated accident-video recording. These results move the paper beyond implementation readiness and provide an initial quantitative baseline, while also showing that future work must reduce missed detections, improve stream continuity during recording, strengthen privacy controls, and validate the system in more realistic field conditions.

ACKNOWLEDGMENT

The authors acknowledge Eng. Job Acosta, instructor of Embedded Systems and Hardware Description Language at the Universidad Evangélica de El Salvador, Faculty of Engineering, for academic guidance. The authors also thank Jorge

Romero and Francisco Reyes, students who participated in prototype testing.

REFERENCES

- [1] U.S. Bureau of Labor Statistics, “National Safety Stand-Down highlights fall hazards in construction,” *The Economics Daily*, Apr. 28, 2026. [Online]. Available: <https://www.bls.gov/opub/ted/2026/national-safety-stand-down-highlights-fall-hazards-in-construction.htm>
- [2] Occupational Safety and Health Administration, “OSHA’s Fall Prevention Campaign,” U.S. Department of Labor, 2026. [Online]. Available: <https://www.osha.gov/stop-falls>. Accessed: Jun. 12, 2026.
- [3] National Institute for Occupational Safety and Health, “Construction Falls: Progress and Prevention,” Centers for Disease Control and Prevention, Apr. 20, 2026. [Online]. Available: <https://www.cdc.gov/niosh/bulletin/2026/falls-stand-down-2026.html>. Accessed: Jun. 12, 2026.
- [4] A. Ramachandran and A. Karuppiyah, “A Survey on Recent Advances in Wearable Fall Detection Systems,” *BioMed Research International*, vol. 2020, Art. no. 2167160, 2020, doi: 10.1155/2020/2167160.
- [5] F. Wu, H. Zhao, Y. Zhao, and H. Zhong, “Development of a Wearable-Sensor-Based Fall Detection System,” *International Journal of Telemedicine and Applications*, vol. 2015, Art. no. 576364, 2015, doi: 10.1155/2015/576364.
- [6] J. C. Ruiz-Garcia, R. Tolosana, R. Vera-Rodríguez, and C. Moro, “CareFall: Automatic Fall Detection through Wearable Devices and AI Methods,” in *Proc. Workshop on Advances of Mobile and Wearable Biometrics (WAMWB 2023)*, CEUR Workshop Proc., vol. 3517, pp. 51–57, 2023. [Online]. Available: <https://ceur-ws.org/Vol-3517/paper4.pdf>
- [7] Z. Qian *et al.*, “Development of a Real-Time Wearable Fall Detection System in the Context of Internet of Things,” *IEEE Internet of Things Journal*, vol. 9, no. 21, pp. 21999–22007, Nov. 2022, doi: 10.1109/IJOT.2022.3181701.
- [8] I. Campero-Jurado, S. Márquez-Sánchez, J. Quintanar-Gómez, S. Rodríguez, and J. M. Corchado, “Smart Helmet 5.0 for Industrial Internet of Things Using Artificial Intelligence,” *Sensors*, vol. 20, no. 21, Art. no. 6241, 2020, doi: 10.3390/s20216241.
- [9] TDK InvenSense, “MPU-6000 and MPU-6050 Product Specification, Revision 3.4,” Aug. 19, 2013. [Online]. Available: <https://web.archive.org/web/20200718085256/https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf>. Accessed: Jun. 12, 2026.
- [10] Espressif Systems, “ESP-NOW,” ESP-IDF Programming Guide, v5.4, 2025. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/v5.4/esp32/api-reference/network/esp_now.html. Accessed: Jun. 12, 2026.
- [11] Espressif Systems, “ESP-NOW,” ESP-FAQ Latest Documentation, 2026. [Online]. Available: <https://docs.espressif.com/projects/esp-faq/en/latest/application-solution/esp-now.html>. Accessed: Jun. 12, 2026.
- [12] Socket.IO, “Server API,” Socket.IO Documentation, 2026. [Online]. Available: <https://socket.io/docs/v4/server-api/>. Accessed: Jun. 12, 2026.
- [13] Supabase, “Storage,” Supabase Documentation, 2026. [Online]. Available: <https://supabase.com/docs/guides/storage>. Accessed: Jun. 12, 2026.
- [14] Supabase, “Storage Buckets: Fundamentals,” Supabase Documentation, 2026. [Online]. Available: <https://supabase.com/docs/guides/storage/buckets/fundamentals>. Accessed: Jun. 12, 2026.